

LOYALTY BRIDGE DESIGN

Connecting the Booking System to ArtiZone Rewards & Loyalty Program

Context

You are building a loyalty/rewards program for ArtiZone (separate initiative). This document defines how the booking system feeds into that program — points earned on visits, redemption at checkout, and unified customer profiles.

Core Principle

A completed appointment is a loyalty event. Every status='completed' appointment triggers points earning. Every booking attempt checks for available rewards. The two systems share the customer table but maintain separate loyalty logic.

Data Model Extension

New Tables (Add to Booking Schema)

```
-- =====
-- LOYALTY CORE TABLES
-- =====

-- Loyalty program configuration
CREATE TABLE loyalty_config (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  points_per_jod numeric(10,2) NOT NULL DEFAULT 1.0, -- 1 JOD spent = X points
  min_redeem_points int NOT NULL DEFAULT 100,        -- minimum points to redeem
  point_value_jod  numeric(10,2) NOT NULL DEFAULT 0.01, -- 1 point = X JOD (1% default)
  welcome_bonus    int NOT NULL DEFAULT 50,          -- points for new customer signup
  referral_bonus   int NOT NULL DEFAULT 100,         -- points for referring a friend
  birthday_bonus   int NOT NULL DEFAULT 200,         -- points on birthday month
  active           boolean NOT NULL DEFAULT true,
  updated_at       timestampz NOT NULL DEFAULT now()
);

-- Customer loyalty profile (extends customer table)
CREATE TABLE customer_loyalty (
  customer_id  uuid PRIMARY KEY REFERENCES customer(id),
  total_points int NOT NULL DEFAULT 0, -- lifetime earned
  available_points int NOT NULL DEFAULT 0, -- current balance (earned - redeemed - expired)
  tier          text NOT NULL DEFAULT 'bronze', -- bronze | silver | gold | platinum
  tier_updated_at timestampz,
  referral_code text UNIQUE, -- e.g. "AZ-RANA-001"
  referred_by   uuid REFERENCES customer(id), -- who referred this customer
  birthday      date,
```

```

    created_at      timestampz NOT NULL DEFAULT now()
);

-- Points transaction log (every point movement)
CREATE TABLE loyalty_transaction (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id     uuid NOT NULL REFERENCES customer(id),
    appointment_id  uuid REFERENCES appointment(id), -- linked to visit (or NULL for manual/adjustment)
    type            text NOT NULL, -- 'earn_visit' | 'earn_referral' | 'earn_welcome' | 'earn_birthday'
    ↪ | 'redeem' | 'expire' | 'adjustment' | 'bonus_campaign'
    points          int NOT NULL, -- positive = earned, negative = redeemed
    description     text,
    balance_after   int NOT NULL, -- available_points after this transaction
    created_by      text, -- 'system' | staff_name | 'campaign_id'
    created_at      timestampz NOT NULL DEFAULT now()
);

-- Tier thresholds
CREATE TABLE loyalty_tier (
    name           text PRIMARY KEY, -- bronze | silver | gold | platinum
    min_points     int NOT NULL, -- minimum lifetime points
    multiplier      numeric(3,2) NOT NULL DEFAULT 1.0, -- points multiplier (gold = 1.5x)
    benefits       text[], -- e.g. ['10% off', 'priority booking', 'free add-on']
    color          text -- UI color code
);

-- Redemption rules (what can be redeemed)
CREATE TABLE loyalty_reward (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    name           text NOT NULL, -- e.g. "Free Hydrafacial"
    description     text,
    points_cost     int NOT NULL, -- points required
    service_id     uuid REFERENCES service(id), -- NULL = generic discount
    discount_jod    numeric(10,2), -- if generic discount
    discount_percent int, -- if percentage discount
    active         boolean NOT NULL DEFAULT true,
    valid_from     timestampz,
    valid_to       timestampz,
    usage_limit     int, -- max redemptions per customer (NULL = unlimited)
    created_at     timestampz NOT NULL DEFAULT now()
);

-- Customer's redeemed rewards (vouchers)
CREATE TABLE customer_reward (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id     uuid NOT NULL REFERENCES customer(id),
    reward_id       uuid NOT NULL REFERENCES loyalty_reward(id),
    points_spent    int NOT NULL,
    status         text NOT NULL DEFAULT 'active', -- active | used | expired
    appointment_id  uuid REFERENCES appointment(id), -- when it was used
    redeemed_at    timestampz NOT NULL DEFAULT now(),

```

```

    used_at          timestamptz,
    expires_at       timestamptz
);

-- Campaign/bonus points (admin can run promotions)
CREATE TABLE loyalty_campaign (
    id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    name              text NOT NULL,
    description        text,
    bonus_points       int NOT NULL,      -- extra points per visit
    multiplier          numeric(3,2) NOT NULL DEFAULT 1.0, -- e.g. 2x points
    service_id         uuid REFERENCES service(id), -- NULL = all services
    min_spend_jod       numeric(10,2),    -- minimum appointment value
    valid_from         timestamptz NOT NULL,
    valid_to           timestamptz NOT NULL,
    active             boolean NOT NULL DEFAULT true,
    created_by         text,
    created_at         timestamptz NOT NULL DEFAULT now()
);

```

Seed Data for Loyalty

```

-- Tier configuration
INSERT INTO loyalty_tier (name, min_points, multiplier, benefits, color) VALUES
    ('bronze', 0, 1.00, ARRAY['Earn points on every visit'], '#CD7F32'),
    ('silver', 500, 1.25, ARRAY['Earn points on every visit', '5% off all services'], '#C0C0C0'),
    ('gold', 1500, 1.50, ARRAY['Earn points on every visit', '10% off all services', 'Priority
    ⇨ booking'], '#FFD700'),
    ('platinum', 5000, 2.00, ARRAY['Earn points on every visit', '15% off all services', 'Priority
    ⇨ booking', 'Free birthday treatment'], '#E5E4E2');

-- Default program config
INSERT INTO loyalty_config (points_per_jod, min_redeem_points, point_value_jod, welcome_bonus,
    ⇨ referral_bonus, birthday_bonus)
VALUES (1.0, 100, 0.01, 50, 100, 200);

-- Sample rewards
INSERT INTO loyalty_reward (name, description, points_cost, service_id, discount_jod, discount_percent,
    ⇨ active, valid_from, valid_to) VALUES
    ('Free Hydrafacial', 'Redeem for a complimentary Hydrafacial session', 4500, (SELECT id FROM service
    ⇨ WHERE name='Hydrafacial'), NULL, NULL, true, now(), now() + interval '1 year'),
    ('Free Laser Session', 'Redeem for a complimentary laser session', 3500, (SELECT id FROM service
    ⇨ WHERE name='Laser session'), NULL, NULL, true, now(), now() + interval '1 year'),
    ('10 JOD Discount', '10 JOD off any service', 1000, NULL, 10.00, NULL, true, now(), now() + interval
    ⇨ '1 year'),
    ('20% Off Any Service', '20% discount on your next visit', 2000, NULL, NULL, 20, true, now(), now()
    ⇨ + interval '6 months');

```

Integration Points

Point 1: Customer Registration → Welcome Bonus

Trigger: New customer created via booking form or walk-in registration

```
-- After INSERT INTO customer...
INSERT INTO customer_loyalty (customer_id, total_points, available_points, referral_code)
VALUES (
    NEW.id,
    (SELECT welcome_bonus FROM loyalty_config WHERE active = true),
    (SELECT welcome_bonus FROM loyalty_config WHERE active = true),
    'AZ-' || UPPER(SUBSTRING(MD5(NEW.id::text), 1, 6))
);

INSERT INTO loyalty_transaction (customer_id, type, points, description, balance_after, created_by)
VALUES (
    NEW.id,
    'earn_welcome',
    (SELECT welcome_bonus FROM loyalty_config WHERE active = true),
    'Welcome bonus for joining ArtiZone Rewards',
    (SELECT welcome_bonus FROM loyalty_config WHERE active = true),
    'system'
);
```

Frontend: Show “Welcome! You’ve earned 50 points” on first booking confirmation.

Point 2: Appointment Completed → Points Earned

Trigger: Appointment status changes to ‘completed’

```
CREATE OR REPLACE FUNCTION earn_points_on_completion()
RETURNS TRIGGER AS $$
DECLARE
    config loyalty_config%ROWTYPE;
    tier_rec loyalty_tier%ROWTYPE;
    service_price numeric(10,2);
    base_points int;
    multiplier numeric(3,2);
    final_points int;
    campaign_bonus int := 0;
    campaign_mult numeric(3,2) := 1.0;
BEGIN
    -- Only process on status change to 'completed'
    IF NEW.status != 'completed' OR OLD.status = 'completed' THEN
        RETURN NEW;
    END IF;

    -- Get active config
    SELECT * INTO config FROM loyalty_config WHERE active = true LIMIT 1;
    IF NOT FOUND THEN RETURN NEW; END IF;
```

```

-- Get service price
SELECT price INTO service_price FROM service WHERE id = NEW.service_id;

-- Get customer tier
SELECT t.* INTO tier_rec
FROM customer_loyalty cl
JOIN loyalty_tier t ON cl.tier = t.name
WHERE cl.customer_id = NEW.customer_id;

-- Calculate base points: price * points_per_jod
base_points := ROUND(service_price * config.points_per_jod);

-- Apply tier multiplier
multiplier := COALESCE(tier_rec.multiplier, 1.0);

-- Check for active campaigns
SELECT COALESCE(SUM(bonus_points), 0), COALESCE(MAX(multiplier), 1.0)
INTO campaign_bonus, campaign_mult
FROM loyalty_campaign
WHERE active = true
      AND valid_from <= now() AND valid_to >= now()
      AND (service_id IS NULL OR service_id = NEW.service_id)
      AND (min_spend_jod IS NULL OR min_spend_jod <= service_price);

-- Final calculation
final_points := ROUND(base_points * multiplier * campaign_mult) + campaign_bonus;

-- Update customer loyalty
UPDATE customer_loyalty
SET total_points = total_points + final_points,
    available_points = available_points + final_points
WHERE customer_id = NEW.customer_id;

-- Log transaction
INSERT INTO loyalty_transaction (customer_id, appointment_id, type, points, description,
    ⇨ balance_after, created_by)
VALUES (
    NEW.customer_id,
    NEW.id,
    'earn_visit',
    final_points,
    'Points earned for ' || (SELECT name FROM service WHERE id = NEW.service_id) ||
    ' (Base: ' || base_points || ', Tier ' || tier_rec.name || ' x' || multiplier ||
    CASE WHEN campaign_bonus > 0 THEN ', Campaign bonus: ' || campaign_bonus ELSE '' END || ')',
    (SELECT available_points FROM customer_loyalty WHERE customer_id = NEW.customer_id),
    'system'
);

-- Check tier upgrade
UPDATE customer_loyalty

```

```

SET tier = (
    SELECT name FROM loyalty_tier
    WHERE min_points <= (SELECT total_points FROM customer_loyalty WHERE customer_id =
        ⇨ NEW.customer_id)
    ORDER BY min_points DESC LIMIT 1
),
tier_updated_at = CASE
    WHEN tier != (SELECT name FROM loyalty_tier WHERE min_points <= (SELECT total_points FROM
        ⇨ customer_loyalty WHERE customer_id = NEW.customer_id) ORDER BY min_points DESC LIMIT 1)
    THEN now() ELSE tier_updated_at END
WHERE customer_id = NEW.customer_id;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_earn_points_on_completion
    AFTER UPDATE OF status ON appointment
    FOR EACH ROW
    EXECUTE FUNCTION earn_points_on_completion();

```

Example:

- Customer: Gold tier (1.5x multiplier)
- Service: Hydrafacial (45 JOD)
- Campaign: “Summer Glow” — 2x points + 50 bonus
- Calculation: $45 * 1.0 * 1.5 * 2.0 + 50 = 185$ points

Point 3: Booking Screen → Show Available Rewards

API Endpoint: GET /api/customers/:id/loyalty

Response:

```

{
  "customer_id": "cust_001",
  "tier": {
    "name": "gold",
    "multiplier": 1.50,
    "benefits": ["Earn points on every visit", "10% off all services", "Priority booking"],
    "color": "#FFD700",
    "next_tier": {
      "name": "platinum",
      "points_needed": 3500
    }
  },
  "points": {
    "available": 2450,
    "total_lifetime": 3200,
    "pending": 0
  }
}

```

```

},
"active_rewards": [
  {
    "id": "cr_001",
    "reward_name": "Free Hydrafacial",
    "points_cost": 4500,
    "status": "active",
    "expires_at": "2026-12-31T00:00:00Z"
  }
],
"can_redeem_now": [
  {
    "reward_id": "lr_003",
    "name": "10 JOD Discount",
    "points_cost": 1000,
    "savings_jod": 10.00
  }
],
"referral_code": "AZ-RANA-001",
"referral_url": "https://artizonespa.com/?ref=AZ-RANA-001"
}

```

On Booking Confirm Screen (Admin):

- Show customer tier badge (Gold, Silver, etc.)
- Show available points: “Customer has 2,450 points”
- Dropdown: “Apply reward” — lists `can_redeem_now` rewards
- If reward selected: adjust final price, mark reward as `used`, create transaction

Point 4: Redeem Points at Checkout

API Endpoint: POST `/api/admin/appointments/:id/redeem`

Request:

```

{
  "customer_reward_id": "cr_001",
  "notes": "Customer redeemed Free Hydrafacial"
}

```

Backend Logic:

```

-- 1. Verify reward belongs to customer and is active
-- 2. Update customer_reward status = 'used', used_at = now(), appointment_id = :id
-- 3. Deduct points from customer_loyalty.available_points
-- 4. Log negative loyalty_transaction
-- 5. Update appointment.price = 0 (or discounted amount)
-- 6. Log appointment_event: price_adjusted

```

Response:

```
{
  "success": true,
  "appointment_id": "appt_001",
  "original_price": 45.00,
  "final_price": 0.00,
  "points_redeemed": 4500,
  "reward_name": "Free Hydrafacial",
  "remaining_points": 1950
}
```

Point 5: Referral Program

Flow:

1. Existing customer gets referral code: AZ-RANA-001
2. New customer books with ?ref=AZ-RANA-001 in URL
3. System stores `referred_by` in `customer_loyalty`
4. After new customer's first completed appointment:
 - Referrer gets 100 points (`referral_bonus`)
 - New customer gets 50 points (`welcome_bonus` + `referral boost`)

API: GET `/api/referral/:code` — validates code, returns referrer name

Point 6: Birthday Bonus

Cron job (daily at 9 AM):

```
SELECT customer_id
FROM customer_loyalty
WHERE EXTRACT(MONTH FROM birthday) = EXTRACT(MONTH FROM now())
  AND EXTRACT(DAY FROM birthday) = EXTRACT(DAY FROM now())
  AND NOT EXISTS (
    SELECT 1 FROM loyalty_transaction
    WHERE customer_id = customer_loyalty.customer_id
      AND type = 'earn_birthday'
      AND EXTRACT(YEAR FROM created_at) = EXTRACT(YEAR FROM now())
  );
```

For each: Add birthday bonus points, send WhatsApp: “Happy Birthday! Enjoy 200 bonus points + a free add-on on your next visit!”

Admin Loyalty Dashboard (New Screen)

Screen: Loyalty Overview

- **Metric cards:** Total members | Points issued (month) | Points redeemed | Active rewards | Top referrers
- **Member search:** By phone, name, tier
- **Member detail:** Points history, tier progress, referral tree, active rewards

Screen: Campaign Manager

- Create/edit bonus point campaigns
- Set: name, bonus points, multiplier, service filter, date range
- Toggle active/inactive
- View campaign performance (appointments triggered, points issued)

Screen: Reward Catalog

- Add/edit rewards
- Set: name, points cost, service link or generic discount
- Active/expire rewards
- View redemption stats

Screen: Breakage Report

- Unredeemed points liability
- Points expiring next 30/60/90 days
- Expired points summary

WhatsApp Messages for Loyalty

Event	Template	Variables
Welcome bonus	loyalty_welcome	{name}, {points}
Points earned	loyalty_earned	{name}, {service}, {points}, {balance}, {tier}
Tier upgrade	loyalty_tier_up	{name}, {new_tier}, {benefits}
Reward reminder	loyalty_reward_remind	{name}, {reward_name}, {points_cost}, {balance}
Birthday bonus	loyalty_birthday	{name}, {points}, {benefits}
Referral success	loyalty_referral	{name}, {friend_name}, {points}, {referral_code}
Points expiring	loyalty_expire_warn	{name}, {points}, {expiry_date}

API Endpoints (Loyalty Module)

Method	Endpoint	Purpose
GET	/api/customers/:id/loyalty	Customer' s full loyalty profile
POST	/api/admin/loyalty/award	Manual points adjustment (staff override)
POST	/api/admin/loyalty/campaigns	Create new campaign
PUT	/api/admin/loyalty/campaigns/:id	Edit/toggle campaign
GET	/api/admin/loyalty/campaigns	List all campaigns
POST	/api/admin/loyalty/rewards	Create new reward
GET	/api/admin/loyalty/rewards	List reward catalog
GET	/api/admin/loyalty/reports/ breakage	Unredeemed points liability
GET	/api/admin/loyalty/reports/ top-customers	Top customers by visits/spend
GET	/api/admin/loyalty/reports/ campaigns	Campaign performance

Integration Sequence

Phase 1 (MVP): Booking system live

↓

Phase 2: Add customer_loyalty table + welcome bonus

↓

Phase 3: Add points earning on completion (trigger)

↓

Phase 4: Add reward catalog + redemption at checkout

↓

Phase 5: Add referral program + campaigns

↓

Phase 6: Add admin dashboard + reporting

Key Decisions

Decision	Rationale
Separate customer_loyalty table	Booking system can run without loyalty; loyalty can be added later
Points per JOD (not per visit)	Rewards higher-value treatments fairly
Tier multipliers (not fixed points)	Incentivizes repeat visits to reach higher tiers
Points expire?	No —simpler for MVP; add 12-month expiry in Phase 2 if needed
Generic rewards (discount JOD/%)	Allows flexibility beyond service-specific rewards
Campaign system	Admin can run “2x points weekends” without code changes

Files for Developer

1. SQL schema extension (loyalty tables + triggers)
2. API endpoint specifications
3. WhatsApp template text (Arabic + English)
4. Admin dashboard wireframes
5. Customer-facing loyalty card design (from Claude.ai reference)